SPACEBOX

Server & Dashboard Guide

Game version 0.1.5

1 Start a server
2 The config file
3 Join the server
4 Open it to the internet
5 Automatic HTTPS
6 The web dashboard
7 Console commands
8 Files, saves and backups
9 Troubleshooting

1. Start a server

The game, the host mode and the dedicated server are the same program. A dedicated server runs without a window and keeps the world running while nobody plays on that PC.

Windows

1. Open the game folder.
2. Double-click `start-server.bat`. A terminal window opens and shows the log.
3. Allow Spacebox in the Windows Firewall question (also on public networks if you want players from outside).

Linux

1. Unzip the Linux build.
2. Run `./start-server.sh` in a terminal.
3. On a machine without a desktop it uses `xvfb-run` if installed (`sudo apt install xvfb`); most servers also run without it.

On the first start the server creates `serverconfig.json` and a new world in `Worlds/ServerWorld`. When the log shows `World ready`, players can join.

> **Stop the server safely**
> Type `stop` in the terminal (or use Stop in the dashboard). The world is saved first. Closing the window without `stop` loses changes since the last autosave.

Without the scripts: `Spacebox.exe --server` (Windows) or `./Spacebox --server` (Linux).

2. The config file

`serverconfig.json` lies next to the executable. Edit it while the server is stopped (most settings can also be changed live in the dashboard). Missing settings are added with their defaults on the next start.

Setting	Default	Meaning
ServerName	Spacebox Server	Name in the server list
Description	A Spacebox server	Text under the name (up to 600 characters)
IconFile	server-icon.png	PNG or JPEG next to the executable, up to 256 KB, square 64–128 px looks best
LanDiscovery	true	Players in the same network find the server automatically
WhitelistEnabled / Whitelist	false / []	Only listed names may join
RegistrationEnabled	false	Players need a password (created on their first join)

Admins	[]	Player names with admin rights: they run every console command from the in-game chat with `/` (e.g. `/gm 1 Anna`, `/kick Bob`). Only works with `RegistrationEnabled`, so nobody can take an admin's name
ShowAdmins	true	Show `[ADMIN]` before admin names in the player list and in chat
WorldName	ServerWorld	Folder in `Worlds/`; a missing world is created
Seed	random	World seed for a new world
ModId	default	GameSet (content pack) of the world
Port	7777	UDP port of the game (the dashboard uses TCP on the same number)
MaxPlayers	16	Player limit
GameMode	0	0 Survival, 1 Creative, 2 Spectator (for a new world)
AutoSaveInterval	300	Seconds between autosaves, 0 turns it off (minimum 10)
UpdateRate	30	Server frame rate, 10–240
FriendlyMobs	false	Mobs never attack first; a mob that is hurt fights back for 30 seconds
DashboardEnabled	true	Web dashboard on or off
DashboardPort	0	TCP port of the dashboard, 0 = same number as `Port`
PublicAddress	empty	The address people use from the internet: a DuckDNS name, any domain or your public IP. Turns on automatic HTTPS (chapter 5)
HttpsAutomatic	true	Get and renew the HTTPS certificate automatically when `PublicAddress` is set
HttpsChallengePort	80	Port for the Let's Encrypt check when no DNS token is set
DuckDnsToken	empty	For `*.duckdns.org` addresses: no port 80 needed, and the server keeps the name on your current IP
DuckDnsUpdateIp	true	Update the DuckDNS IP every 5 minutes
CloudflareApiToken	empty	For domains managed by Cloudflare: no port 80 needed
LetsEncryptEmail	empty	Optional, Let's Encrypt sends expiry warnings there

LetsEncryptStaging	false	Test certificates (not trusted by browsers), only for testing the setup
DashboardCertificateFile / ...KeyFile / ...Password	empty	Your own certificate files instead (PEM + key, or PFX + password); turns the automatic mode off

3. Join the server

- **Same network:** Multiplayer menu → the server appears under local servers.
- **By address:** add a server with the IP (or DuckDNS name) and port 7777.
- **Same PC:** address `127.0.0.1`.

Players get the server's GameSet automatically if they do not have it. The game version must be the same on server and clients.

4. Open it to the internet

1. **Fixed LAN address:** give the server PC a fixed local IP in your router (DHCP reservation), e.g. `192.168.1.50`.
2. **Port forwarding:** in the router forward **UDP 7777** (game) and **TCP 7777** (dashboard) to that IP. Some routers have one rule for "TCP/UDP".
3. **Firewall:** Windows: allow Spacebox when asked, or add inbound rules for TCP and UDP 7777. Linux with ufw: `sudo ufw allow 7777`.
4. **A name instead of an IP:** home IPs change. A free DuckDNS name is easiest (chapter 5): the server keeps it up to date by itself. Any other name service works too.

> **CGNAT**
> If the WAN/Internet IP in your router differs from the IP that duckdns.org shows for you, your provider uses CGNAT and port forwarding cannot work. Ask the provider for a public IPv4 address, or use a VPN/tunnel service (for example a VPS or a mesh VPN for your friends).

> **Inside your own network**
> Many routers cannot open your own public name from inside (no "NAT loopback"). At home use the LAN address, e.g. `192.168.1.50:7777`; friends outside use the DuckDNS name.

5. Automatic HTTPS

The dashboard asks for your admin password. Over the internet it must be encrypted (HTTPS), otherwise the password can be read on the way. The server gets a free certificate from Let's Encrypt and renews it by itself. You only tell it the address people use, and pick one of the ways below.

Your situation	What to put in serverconfig.json	Extra port
Free DuckDNS name	`PublicAddress` + `DuckDnsToken`	none
Own domain on Cloudflare	`PublicAddress` + `CloudflareApiToken`	none

Any other name (No-IP, Dynu, own domain, ...)	`PublicAddress`	TCP 80
Only your public IP	`PublicAddress` = the IP	TCP 80

A. DuckDNS (easiest, free)

1. Open `https://www.duckdns.org`, sign in and add a sub domain, for example `myserver`.
2. Copy the **token** shown at the top of the page.
3. Stop the server and set:

```
"PublicAddress": "myserver.duckdns.org",
"DuckDnsToken": "a1b2c3d4-....-your-token"
```

4. Start the server. The server also keeps the name on your current IP, so the DuckDNS updater program is not needed.

B. Your own domain on Cloudflare

1. In Cloudflare, add a DNS record (A) like `play.example.com` → your public IP, proxy **off** (grey cloud).
2. My Profile → API Tokens → Create Token → template **Edit zone DNS**, zone = your domain. Copy the token.
3. Set:

```
"PublicAddress": "play.example.com",
"CloudflareApiToken": "your-cloudflare-token"
```

C. Any other name or just your IP

1. Make sure the name points to your public IP (No-IP, Dynu and others have their own small updater program).
2. Forward **TCP 80** to the server PC as well (Let's Encrypt checks a file there; the server opens port 80 only for those few seconds).
3. Set:

```
"PublicAddress": "play.example.com"     or your IP, e.g. "85.10.20.30"
```

> **Port 80 not possible?**
> Some providers block it, and on Linux only root may open ports below 1024. Forward external port 80 to another internal port in the router (for example 8080) and set `"HttpsChallengePort": 8080`. If your provider blocks port 80 completely, use DuckDNS or Cloudflare instead.

> **Only an IP address**
> Let's Encrypt gives certificates for IP addresses that last about 6 days; the server renews them automatically every few days. If your IP changes, a name (DuckDNS) is the better choice.

Then

Start the server. After about a minute the log shows `HTTPS certificate for … saved`, and the dashboard overview shows the HTTPS state. Open `https://<your address>:7777/dashboard` from anywhere.

> **What happens in the background**
> The certificate is stored in `dashboard/certificates/`, checked every 12 hours and renewed when a third of its time is left (30 days for normal certificates). Failures are written to the log with a hint and retried after an hour. Set `"LetsEncryptStaging": true` for a first test run: Let's Encrypt then issues test certificates (browsers warn about them) and failed attempts do not count against its limits.

> **Keep tokens secret**
> DuckDNS and Cloudflare tokens control your name. Do not share `serverconfig.json` or screenshots of it.

While HTTPS is active, plain `http://` still works inside your own network; from the internet it is redirected to `https://`. Using your own certificate instead: set `DashboardCertificateFile` and `DashboardCertificateKeyFile`.

6. The web dashboard

First start: create the admin

1. Start the server. The log says: *No admin yet. Register in the dashboard with the setup key from …/dashboard/setup-key.txt.*
2. Open the dashboard: on the server PC `http://localhost:7777/dashboard`, in your network `http://192.168.1.50:7777/dashboard`, from outside `https://<your address>:7777/dashboard`.
3. Open `dashboard/setup-key.txt` in the server folder and copy the key (like `ABCDE-FGHJK-LMNPQ-RSTUV`).
4. Enter the key, an admin name and a password (at least 8 characters). The key file is deleted; only one admin exists.

> **Forgot the password?**
> Stop the server, delete `dashboard/admin.json` and start again: a new setup key is written and you can register again.

Pages

Page	What you can do
Overview	Players, uptime, CPU, memory, game ticks, update time, world size, live charts; server info and HTTPS state; send a chat message to everyone; Save world, Restart, Stop
Players	Online players with ping, address and position; all players who ever joined; Kick, Mute, Ban. Click a player for health, power, oxygen, equipment, hotbar, inventory with item icons and statistics
Moderation	Banned names and IP addresses (unban), muted players, ban by name, whitelist on/off, add and remove names; admins: add and remove, show or hide the [ADMIN] tag
Console	Live server log; type any console command (Tab completes, arrow keys browse the history)

Settings	Name, description, max players, LAN visibility, autosave interval, frame rate (applied at once); upload the server icon; change the dashboard password

> **Security built in**
> Login sessions last 12 hours; 5 wrong passwords lock that address for 5 minutes; every admin action is written to the server log as `[Dashboard] admin: …`. If you never need the dashboard from outside, simply do not forward TCP 7777, or set `"DashboardEnabled": false`.

7. Console commands

Type them in the server terminal or the dashboard console. `help` lists all of them. Admins (see `Admins` in chapter 2) type them in the game chat with a leading `/`; the answer appears only in their own chat. Other players cannot run server commands and cannot change their own game mode. Pressing `/` in the game opens the chat with `/` already typed; `/help` lists the commands of the game client. While typing, the chat suggests commands, player names, item and mob ids: **Tab** completes, **Up/Down** picks a suggestion or browses earlier messages. In chat the player name is optional for `give`, `heal`, `health`, `power`, `air`, `gm` and `mob`: without it the command applies to yourself.

Command	Meaning
say <text>	Chat message from the server
players	List online players
kick <name>	Disconnect a player
ban / unban <name>	Ban (also offline players and their IP) or unban
mute / unmute <name>	Block or allow chat for a player
unregister <name>	Delete a player's password account (registration mode)
gm <0|1|2> <name>	Game mode: survival, creative, spectator. In chat an admin can leave out the name to change their own mode: `/gm 1`
tp <name> <x> <y> <z>	Teleport a player to a position
tpp <name> <target>	Teleport a player to another player
mob <id> <count> <name> | list | clear	Spawn, list or remove mobs
give <item> [count] <name>	Give items (ids like `iron_ingot`; `items [filter]` lists them)
heal <name>	Restore health, power and air
health / power / air [value] <name>	Restore one value to full, or set it
save / stop / restart	Save the world; save and stop; save and restart
clear, version, help	Clear the console, show the version, list commands

8. Files, saves and backups

Path	Content

serverconfig.json	Server settings
Worlds/<WorldName>/	The world: sectors and asteroids, `players/` (one file per player), `moderation.json` (bans, mutes), `accounts.json` (passwords in registration mode)
Worlds/_backups/	Zip backups made automatically before a world is converted to a new game version
dashboard/	`admin.json` (dashboard login), `setup-key.txt` (only before the first registration), `certificates/` (automatic HTTPS)
GameSets/	Content packs (blocks, items, recipes)

> **Backups**
> Stop the server (or run `save`) and copy the world folder. Updating the game: copy the new game files over the old ones, keep `serverconfig.json`, `Worlds/` and `dashboard/`. Worlds from 0.1.2 are converted automatically on the first start (a backup is made first); older worlds cannot be converted.

9. Troubleshooting

Problem	What to check
Friends cannot join	UDP 7777 forwarded to the right LAN IP; firewall rule; same game version; not behind CGNAT (chapter 4)
Dashboard does not open	TCP 7777 forwarded and allowed; you added `:7777` to the address; at home use the LAN IP
"Your connection is not private"	HTTPS is not ready yet or you opened `https://` with an IP: use the DuckDNS name; check the log for certificate errors
Certificate fails	The log line says why and what to check: DuckDNS/Cloudflare token, the name pointing to your public IP, TCP 80 forwarded (or `HttpsChallengePort`). `PublicAddress` must be public, not 192.168.x.x. It retries every hour; Let's Encrypt allows only a few failed attempts per hour, so test with `LetsEncryptStaging`
"Too many attempts"	5 wrong passwords: wait 5 minutes
Port already in use	Another program (or a second server) uses 7777: stop it or change `Port` / `DashboardPort`
World cannot be loaded	The log names the reason; worlds older than 0.1.2 are not supported